

CHƯƠNG 4

KIẾN TRÚC FRONTEND HIỆN ĐẠI VỚI NEXT.JS

Thời lượng: 4 LT | 6 TH — Công nghệ: Next.js 15, TanStack Query v5, Auth.js v5, React Hook Form, Zod, Zustand

MỤC TIÊU CHƯƠNG

Sau khi hoàn thành chương này, sinh viên có khả năng:

Về kiến thức

- Giải thích kiến trúc Next.js App Router: file-based routing, Route Groups, Layout hierarchy, Parallel Routes và cơ chế Loading/Error UI.
- Phân biệt 4 rendering strategies: CSR, SSR, SSG, ISR — biết khi nào dùng chiến lược nào trong ứng dụng thực tế.
- Hiểu sự khác biệt cốt lõi giữa React Server Components (RSC) và Client Components: không gian chạy, khả năng dùng hooks, bundle size.
- Nắm vững cơ chế caching, stale-while-revalidate và Query Keys của TanStack Query v5; hiểu tại sao TanStack Query là lựa chọn cho server state thay vì Zustand hay Context API.
- Trình bày được Auth.js v5 session flow: Credentials Provider, Social Login (Google), JWT vs Database Session, Middleware route protection.
- Hiểu schema-first validation với Zod và cách zodResolver kết nối Zod với React Hook Form; giải thích lợi ích của type inference từ Zod schema.
- Phân biệt rõ Client State (Zustand) và Server State (TanStack Query) — biết loại state nào thuộc về đâu.

Về kỹ năng

- Thiết lập và tổ chức project Next.js 15 với App Router đúng chuẩn: Route Groups, layouts, loading/error boundaries.
- Implement React Server Components lấy dữ liệu trực tiếp trên server; viết Server Actions cho form submission không cần API route.
- Cấu hình TanStack Query Provider, viết custom hooks với useQuery, useMutation (optimistic updates) và useInfiniteQuery.
- Setup Auth.js v5 với Credentials + Google, cấu hình JWT callbacks để gắn access token, bảo vệ routes bằng middleware.
- Xây dựng form phức tạp với React Hook Form + Zod: dynamic field arrays (steps, ingredients), custom validation, error messages tiếng Việt.
- Tối ưu hiển thị ảnh với Next.js Image component: responsive sizes, WebP/AVIF conversion, blur placeholder.

Về thái độ

- Tư duy "render where it makes sense" — không áp dụng cứng nhắc CSR hay SSR cho tất cả trang.
- Hiểu và tôn trọng ranh giới Server/Client boundary — không cố "client-ify" những gì có thể giữ trên server.

GIỚI THIỆU CHƯƠNG

Next.js App Router (giới thiệu từ Next.js 13, ổn định từ 14 và được cải thiện đáng kể trong 15) đánh dấu một sự dịch chuyển cơ bản trong cách React applications được xây dựng: từ mô hình "client-first" sang "server-first". React Server Components cho phép component chạy hoàn toàn trên server, lấy dữ liệu trực tiếp từ database hoặc API mà không cần expose qua HTTP endpoint, giảm JavaScript bundle size đáng kể.

Chương này xây dựng frontend hoàn chỉnh cho Culinary Blog. Chúng ta sẽ kết hợp nhiều công cụ hiện đại: TanStack Query v5 cho server state management (thay thế useEffect + useState phổ biến trước đây), Auth.js v5 cho authentication (kế thừa từ NextAuth.js), React Hook Form + Zod cho form validation type-safe, và Zustand cho UI state. Mỗi công cụ được chọn có mục đích rõ ràng — không có chồng chéo trách nhiệm.

Liên kết với các chương trước: Frontend sẽ gọi đến .NET 10 API đã xây dựng ở Chương 1–3. JWT Authentication từ Chương 2 được tích hợp vào Auth.js session và Axios interceptor. Pagination API từ Chương 1 được consume bởi useInfiniteQuery cho infinite scroll. Recipe Full-Text Search từ Chương 3 được gọi qua TanStack Query.

Frontend

Chương này sử dụng App Router của Next.js 15. Nếu bạn quen với Pages Router (pages/_app.tsx), hãy lưu ý: App Router có kiến trúc hoàn toàn khác — không có getServerSideProps, không có getStaticProps, không có _app.tsx. Thay vào đó là Server Components, Server Actions và layout files.

4.1. NEXT.JS APP ROUTER

App Router là hệ thống routing mặc định của Next.js 13+. Khác với Pages Router (dựa trên file trong /pages), App Router dựa trên thư mục /app. Mỗi segment trong URL tương ứng với một folder; các file đặc biệt (page.tsx, layout.tsx, loading.tsx...) quy định nội dung và hành vi của route đó.

4.1.1 Cấu trúc thư mục Project

Text culinary-blog-web – Cấu trúc App Router

```

culinary-blog-web/
├── app/
│   ├── layout.tsx          ← Root layout (áp dụng cho toàn bộ app)
│   ├── page.tsx            ← Trang chủ: "/" (SSG)
│   ├── not-found.tsx       ← Trang 404 toàn cục
│   ├── globals.css
│   ├── (public)/           ← Route Group – nhóm public routes (không auth)
│   │   ├── recipes/
│   │   │   ├── page.tsx    ← "/recipes" – Danh sách (SSG + ISR)
│   │   │   ├── [slug]/    ← Dynamic segment
│   │   │   │   ├── page.tsx ← "/recipes/pho-bo" – Chi tiết (SSR)
│   │   │   │   └── loading.tsx ← Skeleton UI khi đang load
│   │   │   └── search/
│   │   │       └── page.tsx ← "/recipes/search" – Tìm kiếm (CSR)
│   │   └── categories/
│   │       └── [slug]/
│   │           └── page.tsx ← "/categories/mon-chinh" (ISR)
│   ├── (auth)/             ← Route Group – trang đăng nhập/đăng ký
│   │   ├── login/page.tsx  ← "/login"
│   │   └── register/page.tsx ← "/register"
│   ├── (dashboard)/        ← Route Group – trang cần đăng nhập
│   │   ├── layout.tsx      ← Protected layout (kiểm tra session)
│   │   ├── dashboard/page.tsx ← "/dashboard"
│   │   └── recipes/
│   │       ├── new/page.tsx ← "/recipes/new" – Tạo recipe
│   │       └── [id]/edit/page.tsx
│   └── components/
│       ├── recipe/
│       │   ├── RecipeCard.tsx ← Server Component
│       │   ├── RecipeDetail.tsx ← Server Component
│       │   └── CommentSection.tsx ← 'use client' – TanStack Query
│       └── forms/
│           └── CreateRecipeForm.tsx ← 'use client' – RHF + Zod
├── lib/
│   ├── api/
│   │   ├── axios.ts        ← Axios instance + interceptors
│   │   └── recipes.ts       ← API functions
│   └── auth.ts              ← Auth.js configuration
├── hooks/
│   └── useRecipes.ts        ← TanStack Query custom hooks
├── store/
│   └── useUIStore.ts        ← Zustand UI state
├── types/
│   └── api.ts               ← TypeScript types cho API responses
├── middleware.ts            ← Auth.js route protection middleware
└── next.config.ts
  
```

4.1.2 Các file đặc biệt trong App Router

File	Mục đích	Render
page.tsx	Nội dung route — UI duy nhất được công khai	Server (mặc định)
layout.tsx	Shared UI bao ngoài page — persist khi navigate	Server
loading.tsx	Skeleton UI trong khi page đang stream	Server
error.tsx	Error boundary — bắt lỗi runtime trong subtree	Client (bắt buộc)
not-found.tsx	UI khi notFound() được gọi hoặc route không tồn tại	Server
template.tsx	Như layout nhưng re-mount khi navigate (thường ít dùng)	Server
route.ts	API Route Handler — /api endpoints trong App Router	Server

Bảng 4.1. Các file đặc biệt trong Next.js App Router

4.1.3 Root Layout và Providers Setup

TypeScript/JSX `app/layout.tsx`

```
import type { Metadata } from 'next';
import { Inter } from 'next/font/google';
import { Providers } from './providers';
import './globals.css';

const inter = Inter({ subsets: 'latin', 'vietnamese' });

// Metadata API: thay thế <Head> – type-safe, tree-shakeable
export const metadata: Metadata = {
  title: { template: '%s | Culinary Blog', default: 'Culinary Blog – Ẩm thực Việt' },
  description: 'Khám phá hàng nghìn công thức nấu ăn Việt Nam và thế giới',
  openGraph: {
    type: 'website',
    locale: 'vi_VN',
    siteName: 'Culinary Blog',
  },
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="vi" suppressHydrationWarning>
      <body className={inter.className}>
        { /* Providers là Client Component – wrap toàn bộ app */ }
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

TypeScript/JSX app/providers.tsx

```
'use client'; // Bắt buộc – QueryClientProvider dùng hooks

import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';
import { SessionProvider } from 'next-auth/react';
import { useState } from 'react';

export function Providers({ children }: { children: React.ReactNode }) {
  // useState để QueryClient không bị share giữa các requests (Server-side)
  const [queryClient] = useState(() => new QueryClient({
    defaultOptions: {
      queries: {
        staleTime: 60 * 1000, // 1 phút: data cũ hơn 1' sẽ
        refetch
        gcTime: 5 * 60 * 1000, // 5 phút: cleanup cache sau 5'
        không dùng
        retry: 1, // Retry 1 lần nếu request thất bại
        refetchOnWindowFocus: false, // Không refetch khi user switch
        tab
      },
    },
  }));

  return (
    <SessionProvider>
      <QueryClientProvider client={queryClient}>
        {children}
        {/* DevTools chỉ xuất hiện trong development */}
        <ReactQueryDevtools initialIsOpen={false} buttonPosition="bottom-right"
      />
    </QueryClientProvider>
  </SessionProvider>
  );
}
```

💡 Lưu ý

Route Groups (thư mục có dấu ngoặc như (public), (auth)) không tạo ra URL segment. (public)/recipes/page.tsx có URL là "/recipes", không phải "/public/recipes". Mục đích: nhóm các routes có chung layout hoặc middleware logic mà không thay đổi URL.

4.2. RENDERING STRATEGIES

Next.js App Router hỗ trợ 4 chiến lược rendering. Điều quan trọng là mỗi trang có thể dùng chiến lược khác nhau — không có one-size-fits-all. Lựa chọn đúng chiến lược ảnh hưởng trực tiếp đến Core Web Vitals, SEO và trải nghiệm người dùng.

Chiến lược	Khi nào render	Cache?	Phù hợp với	Ví dụ trong Culinary Blog
CSR — Client-Side Rendering	Mỗi lần user tương tác, trên browser	Không	Dashboard cá nhân, trang tìm kiếm real-time	Search page, comment form
SSR — Server-Side Rendering	Mỗi request, trên server	Không (trừ khi cấu hình)	Data thay đổi thường xuyên, cần user context	Recipe detail (view count, liked state)
SSG — Static Site Generation	Build time, trên server	Có — vĩnh viễn đến khi rebuild	Content ít thay đổi, cần performance cao	Category list, static pages
ISR — Incremental Static Regeneration	Build time + theo chu kỳ/on-demand	Có — tự động hết hạn	Content cập nhật định kỳ	Recipe list (revalidate: 1 giờ)

Bảng 4.2. So sánh 4 Rendering Strategies trong Next.js App Router

4.2.1 SSG + ISR — Recipe List (cập nhật mỗi giờ)

TypeScript/JSX `app/(public)/recipes/page.tsx`

```
import { RecipeCard } from '@components/recipe/RecipeCard';
import { recipeApi } from '@lib/api/server'; // Server-side fetch

// ISR: revalidate mỗi 3600 giây (1 giờ)
// Sau 1 giờ, lần request tiếp theo sẽ trigger re-render trên background
export const revalidate = 3600;

// Metadata tĩnh cho SEO
export const metadata = {
  title: 'Công thức nấu ăn',
  description: 'Khám phá hàng nghìn công thức nấu ăn Việt Nam và thế giới',
};

export default async function RecipesPage({
  searchParams,
}: {
  searchParams: Promise<{ page?: string; category?: string }>;
}) {
  // Trong App Router, searchParams là async trong Next.js 15
  const { page = '1', category } = await searchParams;

  // Fetch trực tiếp trên server – không có useEffect, không có loading state
  const result = await recipeApi.getList({
    page: parseInt(page),
    pageSize: 12,
    categoryId: category,
  });

  return (
    <main className="container mx-auto px-4 py-8">
```

```

    <h1 className="text-3xl font-bold mb-8">Công thức nấu ăn</h1>
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
      {result.items.map(recipe => (
        <RecipeCard key={recipe.id} recipe={recipe} />
      ))}
    </div>
    { /* Pagination component */ }
  </main>
);
}

```

4.2.2 SSR — Recipe Detail (render theo request)

```

TypeScript/JSX  app/(public)/recipes/[slug]/page.tsx

import { notFound }      from 'next/navigation';
import type { Metadata } from 'next';
import { RecipeDetail }  from '@components/recipe/RecipeDetail';
import { CommentSection } from '@components/recipe/CommentSection';
import { recipeApi }     from '@lib/api/server';

type Props = { params: Promise<{ slug: string }> };

// Dynamic metadata – cần fetch data nên không thể static
export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { slug } = await params;
  const recipe = await recipeApi.getBySlug(slug).catch(() => null);
  if (!recipe) return { title: 'Không tìm thấy' };

  return {
    title: recipe.title,
    description: recipe.description,
    openGraph: {
      title: recipe.title,
      images: recipe.primaryImageUrl ? [recipe.primaryImageUrl] : [],
    },
  };
}

// SSR: không có revalidate, không có generateStaticParams
// → render on every request, luôn có data mới nhất
export default async function RecipeDetailPage({ params }: Props) {
  const { slug } = await params;
  const recipe = await recipeApi.getBySlug(slug).catch(() => null);

  // notFound() ném error được bắt bởi not-found.tsx
  if (!recipe) notFound();

  return (
    <article>
      { /* Server Component: render recipe data từ server */ }
      <RecipeDetail recipe={recipe} />
      { /* Client Component: comments cần interactivity */ }
      <CommentSection recipeId={recipe.id} />
    </article>
  );
}

```

TypeScript/JSX app/(public)/recipes/[slug]/loading.tsx

```
// Loading.tsx: tự động hiển thị khi page đang stream data từ server
// Không cần code gì trong component cha – Next.js tự động dùng Suspense
export default function RecipeDetailLoading() {
  return (
    <div className="container mx-auto px-4 py-8 animate-pulse">
      {/* Skeleton UI */}
      <div className="h-8 bg-gray-200 rounded w-3/4 mb-4" />
      <div className="aspect-video bg-gray-200 rounded-xl mb-6" />
      <div className="space-y-3">
        {[...Array(5)].map((_, i) => (
          <div key={i} className="h-4 bg-gray-200 rounded" />
        ))}
      </div>
    </div>
  );
}
```

Khái niệm

Streaming trong App Router: Next.js tự động wrap mỗi page trong Suspense. Khi loading.tsx tồn tại, server sẽ stream HTML về browser ngay lập tức với loading skeleton, trong khi page.tsx đang fetch data. Điều này cải thiện Time to First Byte (TTFB) và Largest Contentful Paint (LCP) — hai Core Web Vitals quan trọng.

4.3. REACT SERVER COMPONENTS VÀ SERVER ACTIONS

4.3.1 Server Components vs Client Components

React Server Components (RSC) là paradigm mới nhất của React: component chạy hoàn toàn trên server, không xuất hiện trong JavaScript bundle gửi xuống client. RSC có thể fetch data trực tiếp từ database, gọi internal services, đọc filesystem — không giới hạn bởi CORS hay API exposure.

Server Component (Mặc định)

- Chạy trên server — không bao giờ xuống client
- Có thể dùng `async/await` trực tiếp
- Truy cập DB, file system, biến môi trường trực tiếp
- Không thể dùng: `useState`, `useEffect`, hooks
- Không thể dùng browser API: `window`, `document`
- Bundle size = 0 (không gửi JS xuống client)

Client Component ('use client')

- Chạy trên cả server (hydration) VÀ client
- Có thể dùng `useState`, `useEffect`, tất cả hooks
- Có thể dùng browser API, event listeners
- Không thể dùng `async/await` ở top-level component
- Được gửi xuống client trong JS bundle

- Sử dụng cho: UI tĩnh, data display, layouts

- Sử dụng cho: forms, interactive UI, TanStack Query

Hình 4.1. So sánh Server Component và Client Component

4.3.2 Server Component — RecipeDetail

```
TypeScript/JSX components/recipe/RecipeDetail.tsx

// Không có 'use client' → đây là Server Component (mặc định)
import Image from 'next/image';
import { RecipeDto } from '@types/api';

// Props interface: TypeScript enforce type safety
interface RecipeDetailProps { recipe: RecipeDto; }

// Async component – chỉ có trong Server Components
// Dữ liệu được truyền xuống qua props từ page.tsx
export function RecipeDetail({ recipe }: RecipeDetailProps) {
  const totalTime = recipe.prepTimeMinutes + recipe.cookTimeMinutes;

  return (
    <div className="max-w-4xl mx-auto">
      {/* Hero Image – next/image tự optimize WebP/AVIF */}
      <div className="relative aspect-video rounded-2xl overflow-hidden mb-8">
        <Image
          src={recipe.primaryImageUrl ?? '/images/placeholder.jpg'}
          alt={recipe.title}
          fill
          sizes="(max-width: 768px) 100vw, 896px"
          className="object-cover"
          priority // LCP image: Load ngay không Lazy
        />
      </div>

      <h1 className="text-4xl font-bold mb-2">{recipe.title}</h1>
      <p className="text-gray-600 mb-6">{recipe.description}</p>

      {/* Meta info */}
      <div className="grid grid-cols-4 gap-4 mb-8 p-4 bg-orange-50 rounded-xl">
        <Stat label="Chuẩn bị" value={recipe.prepTimeMinutes + ' phút'} />
        <Stat label="Nấu" value={recipe.cookTimeMinutes + ' phút'} />
        <Stat label="Tổng" value={totalTime + ' phút'} />
        <Stat label="Khẩu phần" value={recipe.servings + ' người'} />
      </div>

      {/* Ingredients */}
      <section>
        <h2 className="text-2xl font-semibold mb-4">Nguyên liệu</h2>
        <ul className="space-y-2">
          {recipe.ingredients.map((ing) => (
            <li key={ing.id} className="flex items-center gap-3">
              <span className="font-medium">{ing.quantity}</span> {ing.unit}</span>
              <span>{ing.name}</span>
              {ing.notes && <span className="text-gray-400 text-sm">{ing.notes}</span>}
            </li>
          )}
        </ul>
      </section>
    </div>
  );
}
```

```

        </li>
      )}
    </ul>
  </section>

  { /* Steps */ }
  <section className="mt-8">
    <h2 className="text-2xl font-semibold mb-4">Cách làm</h2>
    <ol className="space-y-6">
      {recipe.steps.map((step) => (
        <li key={step.id} className="flex gap-4">
          <span className="flex-shrink-0 w-8 h-8 bg-orange-500 text-white
rounded-full flex items-center justify-center font-bold">
            {step.stepNumber}
          </span>
          <div>
            <p>{step.description}</p>
            {step.durationMinutes && (
              <p className="text-sm text-gray-400 mt-1">
                ⌚ {step.durationMinutes} phút
              </p>
            )}
          </div>
        </li>
      )}
    </ol>
  </section>
</div>
);
}

function Stat({ label, value }: { label: string; value: string }) {
  return (
    <div className="text-center">
      <div className="text-xl font-bold text-orange-600">{value}</div>
      <div className="text-sm text-gray-500">{label}</div>
    </div>
  );
}

```

4.3.3 Server Actions — Submit form không cần API route

Server Actions là async functions chạy trên server, được gọi từ Client Components như gọi function thông thường. Next.js tự động tạo API endpoint internal — developer không cần viết route handler. Phù hợp cho simple mutations như add comment, like recipe.

```

TypeScript  app/(public)/recipes/[slug]/actions.ts

'use server'; // Đánh dấu file này chứa Server Actions

import { auth } from '@lib/auth';
import { revalidatePath } from 'next/cache';

// Server Action: function async export, chạy trên server
// Được gọi từ Client Component như function bình thường
export async function addCommentAction(

```

```

    recipeId: string,
    formData: FormData,
  ): Promise<{ success: boolean; error?: string }> {
    // Lấy session trực tiếp trên server – không cần JWT từ client
    const session = await auth();
    if (!session?.user) return { success: false, error: 'Chưa đăng nhập.' };

    const content = formData.get('content') as string;
    if (!content?.trim()) return { success: false, error: 'Nội dung không được trống.' };

    try {
      // Gọi API backend (hoặc trực tiếp vào DB nếu full-stack)
      await fetch(`${process.env.API_URL}/recipes/${recipeId}/comments`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${session.user.accessToken}`,
        },
        body: JSON.stringify({ content: content.trim() }),
      });

      // Xóa cache của route này → Next.js sẽ re-fetch dữ liệu
      revalidatePath(`/recipes/${recipeId}`);
      return { success: true };
    } catch {
      return { success: false, error: 'Có lỗi xảy ra. Vui lòng thử lại.' };
    }
  }
}

```

4.4. TANSTACK QUERY (REACT QUERY) V5

TanStack Query (tên cũ: React Query) là thư viện quản lý server state — một loại state đặc biệt: data được fetch từ server, có thể stale (cũ), cần sync với nhiều component, cần caching và background refetching. Đây là vấn đề mà useState + useEffect giải quyết rất kém: không có deduplication, không có caching, dễ bị race conditions và memory leaks.

1 Component mount → kiểm tra cache

queryKey: ["recipes", "detail", "pho-bo"]



2a Cache MISS → fetch từ API

fetch("/api/v1/recipes/pho-bo") → lưu vào cache



2b Cache HIT (fresh < 5 phút) → return ngay

Không fetch mới — user thấy data ngay lập tức



2c Cache HIT (stale > 5 phút) → return stale + refetch background

Stale-While-Revalidate: user thấy data cũ ngay, data mới cập nhật sau



3 Component unmount → GC timer bắt đầu (5 phút)*Sau 5 phút không có consumer → xóa khỏi cache**Hình 4.2. TanStack Query Caching Flow — Stale-While-Revalidate Pattern***4.4.1 Query Keys — Danh tính của cache**

Query Keys là mảng serializable uniquely identify mỗi cache entry. Thiết kế Query Keys cẩn thận giúp invalidation chính xác: khi tạo recipe mới, chỉ cần invalidate recipes.lists() — không ảnh hưởng đến recipe detail cache.

TypeScript lib/api/axios.ts

```
import axios from 'axios';
import { getSession } from 'next-auth/react';

// Tạo Axios instance với base URL từ environment
const apiClient = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL ?? 'https://localhost:5001',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json' },
});

// Request interceptor: tự động gắn JWT vào mọi request
apiClient.interceptors.request.use(async (config) => {
  // getSession() Lấy session từ Auth.js (cookie-based, không expose JWT về client)
  const session = await getSession();
  if (session?.user.accessToken) {
    config.headers.Authorization = `Bearer ${session.user.accessToken}`;
  }
  return config;
});

// Response interceptor: xử lý 401 Unauthorized toàn cục
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Token hết hạn → redirect về login
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default apiClient;
```

TypeScript hooks/useRecipes.ts

```
import {
  useQuery, useMutation, useInfiniteQuery, useQueryClient,
  type InfiniteData,
} from '@tanstack/react-query';
```

```
import apiClient from '@lib/api/axios';
import type { RecipeDto, RecipeListDto, PaginatedResult } from '@types/api';

// — Query Keys – centralized, type-safe —————
// Dùng const object để tránh magic strings, dễ invalidation
export const recipeKeys = {
  all: () => 'recipes' as const,
  lists: () => ...recipeKeys.all(), 'list' as const,
  list: (f: RecipeFilters) => [...recipeKeys.lists(), f] as const,
  details: () => ...recipeKeys.all(), 'detail' as const,
  detail: (slug: string) => [...recipeKeys.details(), slug] as const,
  comments: (id: string) => 'comments', id as const,
};

// — useRecipes – danh sách recipes với filters —————
export function useRecipes(filters: RecipeFilters = {}) {
  return useQuery({
    queryKey: recipeKeys.list(filters),
    queryFn: () => apiClient
      .get<PaginatedResult<RecipeListDto>>('/api/v1/recipes', {
        params: filters })
      .then(r => r.data),
    staleTime: 60 * 1000, // 1 phút – list không cần real-time
  });
}

// — useRecipe – chi tiết một recipe —————
export function useRecipe(slug: string) {
  return useQuery({
    queryKey: recipeKeys.detail(slug),
    queryFn: () => apiClient
      .get<RecipeDto>(`/api/v1/recipes/${slug}`)
      .then(r => r.data),
    staleTime: 5 * 60 * 1000, // 5 phút
    enabled: !!slug, // Không fetch nếu slug undefined
  });
}
```

4.4.2 useMutation với Optimistic Updates

Optimistic Updates cải thiện UX bằng cách update UI ngay lập tức trước khi server xác nhận. Nếu request thất bại, UI tự động roll back. Đây là pattern của các ứng dụng modern: nhấn Like → icon tim đỏ ngay, không phải chờ 200ms.

Text hooks/useRecipes.ts (tiếp theo – useLikeRecipe)

```
// — useLikeRecipe – optimistic update khi like recipe —————
export function useLikeRecipe() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: (recipeId: string) =>
      apiClient.post(`/api/v1/recipes/${recipeId}/like`).then(r => r.data),
```

```

// onMutate: chạy NGAY KHI mutate() được gọi (trước khi request hoàn thành)
onMutate: async (recipeId) => {
  // Hủy bất kỳ refetch đang pending để tránh overwrite
  await queryClient.cancelQueries({ queryKey: recipeKeys.all() });

  // Lưu snapshot của data hiện tại để rollback nếu lỗi
  const previousData =
    queryClient.getQueryData(recipeKeys.detail(recipeId));

  // Cập nhật cache lạc quan – user thấy ngay lập tức
  queryClient.setQueryData(recipeKeys.detail(recipeId), (old: RecipeDto |
    undefined) => {
    if (!old) return old;
    return { ...old, likeCount: old.likeCount + 1, isLiked: true };
  });

  return { previousData }; // Context để dùng trong onError
},

// onError: rollback nếu server trả lỗi
onError: (_error, recipeId, context) => {
  if (context?.previousData) {
    queryClient.setQueryData(recipeKeys.detail(recipeId),
      context.previousData);
  }
},

// onSettled: luôn chạy sau khi success hoặc error
// Refetch để đảm bảo server state và client state đồng bộ
onSettled: (_data, _error, recipeId) => {
  queryClient.invalidateQueries({ queryKey: recipeKeys.detail(recipeId) });
},
});

}

// — useCreateRecipe – tạo recipe mới —————
export function useCreateRecipe() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: (data: CreateRecipeFormData) =>
      apiClient.post<RecipeDto>('/api/v1/recipes', data).then(r => r.data),

    onSuccess: () => {
      // Invalidate toàn bộ list queries → force refetch khi user xem danh sách
      queryClient.invalidateQueries({ queryKey: recipeKeys.lists() });
    },
  });
}

```

4.4.3 useInfiniteQuery — Infinite Scroll Comments

TypeScript hooks/useComments.ts

```

import { useInfiniteQuery } from '@tanstack/react-query';
import apiClient from '@/lib/api/axios';
import { recipeKeys } from './useRecipes';

```

```

export function useInfiniteComments(recipeId: string) {
  return useInfiniteQuery({
    queryKey:      recipeKeys.comments(recipeId),
    queryFn:       ({ pageParam = 1 }) =>
      apiClient
        .get(`/api/v1/recipes/${recipeId}/comments`, {
          params: { page: pageParam, pageSize: 10 },
        })
        .then(r => r.data),

    // Trả về page number tiếp theo, hoặc undefined nếu hết trang
    getNextPageParam: (lastPage) =>
      lastPage.hasNextPage ? lastPage.page + 1 : undefined,

    initialPageParam: 1,
    staleTime:        30 * 1000, // 30 giây
  });
}

// Dùng trong CommentSection component:
// const { data, fetchNextPage, hasNextPage, isFetchingNextPage } =
//   useInfiniteComments(recipeId);
// data.pages.flatMap(p => p.items) → flat list of all loaded comments

```

4.5. AUTH.JS (NEXTAUTH.JS V5)

Auth.js v5 (tên package: next-auth v5) là solution authentication chính thức cho Next.js. So với tự implement JWT trên client, Auth.js quản lý session an toàn hơn: access token KHÔNG được lưu trong localStorage (XSS risk) mà trong HTTP-only cookie và memory. Bên ngoài React tree, session được đọc server-side qua auth().

4.5.1 Luồng Authentication với Auth.js

1 User nhập Email + Password → POST /auth/callback/credentials

Auth.js gọi hàm authorize() của Credentials Provider



2 authorize() gọi Backend API: POST /api/v1/auth/login

Backend xác minh mật khẩu, trả về accessToken + refreshToken



3 Auth.js tạo Session JWT (khác với Backend JWT)

jwt callback gắn accessToken, refreshToken vào Session JWT



4 Session JWT lưu trong HTTP-only cookie (tự động)

Client KHÔNG đọc được cookie này — an toàn hơn localStorage



5 Request kế tiếp: Middleware đọc session từ cookie

Nếu route protected và chưa login → redirect /login



6 Axios interceptor đọc session → gắn accessToken vào header

Authorization: Bearer <backend accessToken>

Hình 4.3. Auth.js v5 Authentication Flow trong Culinary Blog

4.5.2 Cấu hình Auth.js (auth.ts)

```
TypeScript lib/auth.ts

import NextAuth, { type DefaultSession } from 'next-auth';
import Credentials from 'next-auth/providers/credentials';
import Google from 'next-auth/providers/google';

// — Mở rộng Session type để thêm accessToken —————
// TypeScript module augmentation: tránh 'any' cast
declare module 'next-auth' {
  interface Session {
    user: { id: string; accessToken: string; } & DefaultSession['user'];
  }
  interface User {
    accessToken?: string;
    refreshToken?: string;
  }
}

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers:
    // — Credentials Provider – email/password —————
    Credentials({
      credentials: {
        email: { label: 'Email', type: 'email' },
        password: { label: 'Password', type: 'password' },
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) return null;

        try {
          // Gọi Backend .NET API
          const response = await fetch(
            `${process.env.API_URL}/api/v1/auth/login`,
            {
              method: 'POST',
              headers: { 'Content-Type': 'application/json' },
              body: JSON.stringify({
                email: credentials.email,
                password: credentials.password,
              }),
            }
          );

          if (!response.ok) return null;

          const data = await response.json();
          // Trả về User object – sẽ được đưa vào jwt callback
```

```

        return {
          id: data.user.id,
          email: data.user.email,
          name: data.user.fullName,
          image: data.user.avatarUrl,
          accessToken: data.accessToken,
          refreshToken: data.refreshToken,
        };
      } catch {
        return null;
      }
    },
  },
 )),

  // — Google Provider — Social Login —————
  Google({
    clientId: process.env.GOOGLE_CLIENT_ID!,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
  }),
],

callbacks: {
  // jwt: chạy khi tạo/cập nhật JWT session token
  async jwt({ token, user, account }) {
    // Lần đầu sign in: user object có dữ liệu
    if (user) {
      token.accessToken = user.accessToken;
      token.refreshToken = user.refreshToken;
      token.userId = user.id;
    }
    // Google Sign In: Lưu provider access token
    if (account?.provider === 'google') {
      token.googleAccessToken = account.access_token;
    }
    return token;
  },

  // session: chạy khi useSession() hoặc auth() được gọi
  async session({ session, token }) {
    //Expose accessToken lên session để Axios interceptor dùng
    session.user.id = token.userId as string;
    session.user.accessToken = token.accessToken as string;
    return session;
  },
},

pages: {
  signIn: '/login', // Redirect về /Login thay vì /auth/signin mặc định
},
});

```

4.5.3 Middleware — Bảo vệ Routes tự động

TypeScript middleware.ts

```

// File này PHẢI đặt ở root của project (ngang với /app)
import { auth } from './lib/auth';

```

```
import { NextResponse } from 'next/server';

// Middleware chạy TRƯỚC KHI request đến page/API handler
// Perfect cho authentication check – không cần server component riêng
export default auth((req) => {
  const { nextUrl, auth: session } = req;
  const isLoggedIn = !!session;

  // Các paths cần bảo vệ (prefix matching)
  const isProtected =
    nextUrl.pathname.startsWith('/dashboard') ||
    nextUrl.pathname.startsWith('/recipes/new') ||
    nextUrl.pathname.match(/^\/recipes\/^\/]+\/edit/);

  // Redirect về Login nếu chưa đăng nhập
  if (isProtected && !isLoggedIn) {
    const redirectUrl = new URL('/login', req.url);
    redirectUrl.searchParams.set('callbackUrl', nextUrl.pathname);
    return NextResponse.redirect(redirectUrl);
  }

  // Redirect về dashboard nếu đã đăng nhập mà vào /login, /register
  const isAuthPage = nextUrl.pathname.startsWith('/login') ||
    nextUrl.pathname.startsWith('/register');
  if (isAuthPage && isLoggedIn) {
    return NextResponse.redirect(new URL('/dashboard', req.url));
  }

  return NextResponse.next();
});

// matcher: chỉ áp dụng middleware cho các paths này
// Loại trừ static files, next internals, api routes
export const config = {
  matcher: '/((?!api|_next/static|_next/image|favicon.ico|images).*)',
};
```

4.5.4 Đọc Session trong Server và Client Component

Text Sử dụng Session trong Server Component và Client Component

```
// — Trong Server Component (app/dashboard/page.tsx) —————
import { auth } from '@lib/auth';
import { redirect } from 'next/navigation';

export default async function DashboardPage() {
  // auth() đọc session từ cookie trực tiếp trên server
  const session = await auth();
  if (!session) redirect('/login');

  return <h1>Xin chào, {session.user.name}</h1>;
}

// — Trong Client Component ('use client') —————
'use client';
```

```
import { useSession, signOut } from 'next-auth/react';

export function UserMenu() {
  // useSession() hook – đọc session trên client (không cần auth())
  const { data: session, status } = useSession();

  if (status === 'loading') return <div>Đang tải...</div>;
  if (!session) return <a href="/login">Đăng nhập</a>;

  return (
    <div>
      <span>{session.user.name}</span>
      <button onClick={() => signOut({ callbackUrl: '/' })}>
        Đăng xuất
      </button>
    </div>
  );
}
```

4.6. REACT HOOK FORM VÀ ZOD

React Hook Form là thư viện form management tối ưu cho React: không re-render component khi user gõ (uncontrolled approach), tích hợp validation linh hoạt. Zod là schema validation library TypeScript-first: định nghĩa schema → tự động infer TypeScript type → không cần viết interface riêng. Kết hợp qua zodResolver, chúng ta có form validation vừa type-safe vừa runtime-safe.

4.6.1 Zod Schema — Schema là Single Source of Truth

```
TypeScript  types/recipe.ts

import { z } from 'zod';

// — Zod Schema – định nghĩa validation rules
export const createRecipeSchema = z.object({
  title: z.string()
    .min(5, 'Tiêu đề phải có ít nhất 5 ký tự')
    .max(200, 'Tiêu đề không được quá 200 ký tự'),

  description: z.string()
    .min(20, 'Mô tả phải có ít nhất 20 ký tự')
    .max(500, 'Mô tả không được quá 500 ký tự')
    .optional(),

  categoryId: z.string({ required_error: 'Vui lòng chọn danh mục' })
    .uuid('ID danh mục không hợp lệ'),

  prepTimeMinutes: z.number({ invalid_type_error: 'Vui lòng nhập số phút' })
    .int().min(1).max(1440, 'Không hợp lệ (tối đa 24 giờ)'),

  cookTimeMinutes: z.number({ invalid_type_error: 'Vui lòng nhập số phút' })
```

```

    .int().min(1).max(1440),

    servings: z.number().int().min(1).max(50, 'Tối đa 50 khẩu phần'),

    difficulty: z.enum('Easy', 'Medium', 'Hard'], {
      required_error: 'Vui lòng chọn độ khó',
    }),

    // Nested schema cho dynamic array (useFieldArray)
    steps: z.array(z.object({
      description: z.string().min(10, 'Mô tả bước phải có ít nhất 10 ký tự'),
      durationMinutes: z.number().int().min(1).optional(),
    })).min(1, 'Phải có ít nhất 1 bước thực hiện'),

    ingredients: z.array(z.object({
      name: z.string().min(2, 'Tên nguyên liệu tối thiểu 2 ký tự'),
      quantity: z.number({ invalid_type_error: 'Nhập số lượng' }).positive('Phải là số dương'),
      unit: z.string().min(1, 'Nhập đơn vị đo'),
      notes: z.string().optional(),
    })).min(1, 'Phải có ít nhất 1 nguyên liệu'),
  });

  // — TypeScript type từ Zod schema – KHÔNG cần viết interface riêng
  // z.infer<T> tự động tạo type khớp với schema
  export type CreateRecipeFormData = z.infer<typeof createRecipeSchema>;

  // Ví dụ type được infer:
  // type CreateRecipeFormData = {
  //   title: string;
  //   description?: string;
  //   categoryId: string;
  //   difficulty: "Easy" | "Medium" | "Hard";
  //   steps: { description: string; durationMinutes?: number }[];
  //   ingredients: { name: string; quantity: number; unit: string; notes?: string }[];
  //   ...
  // }

```

4.6.2 React Hook Form với Dynamic Field Arrays

```

TypeScript/JSX components/forms/CreateRecipeForm.tsx

'use client';
import { useForm, useFieldArray, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { useRouter } from 'next/navigation';
import { createRecipeSchema, type CreateRecipeFormData } from '@/types/recipe';
import { useCreateRecipe } from '@/hooks/useRecipes';

export function CreateRecipeForm({ categories }: { categories: CategoryDto[] }) {
  const router = useRouter();
  const { mutate: createRecipe, isPending } = useCreateRecipe();

  const {
    register,          // Đăng ký input field với validation

```

```

    control,          // Control object cho useFieldArray, Controller
    handleSubmit,    // Wrapper chạy validate rồi mới gọi onSubmit
    formState: { errors, isSubmitting },
    setError,         // Thêm lỗi từ server-side validation
    watch,            // Theo dõi giá trị field real-time
  } = useForm<CreateRecipeFormData>({
    resolver:         zodResolver(createRecipeSchema), // Kết nối Zod với RHF
    defaultValues: {
      difficulty: 'Medium',
      servings: 4,
      steps: { description: '' }], // Ít nhất 1 step ban đầu
      ingredients: { name: '', quantity: 1, unit: '' }],
    },
  });

// useFieldArray: quản lý dynamic list (thêm/xóa/di chuyển)
const {
  fields: stepFields,
  append: addStep,
  remove: removeStep,
} = useFieldArray({ control, name: 'steps' });

const {
  fields: ingredientFields,
  append: addIngredient,
  remove: removeIngredient,
} = useFieldArray({ control, name: 'ingredients' });

const onSubmit = (data: CreateRecipeFormData) => {
  createRecipe(data, {
    onSuccess: (recipe) => {
      router.push(`/recipes/${recipe.slug}`);
      router.refresh(); // Invalidate Next.js cache
    },
    onError: (error: any) => {
      // Xử lý validation errors từ server (422 Unprocessable Entity)
      if (error.response?.data?.errors) {
        Object.entries(error.response.data.errors).forEach(([field, messages])
=> {
          setError(field as keyof CreateRecipeFormData, {
            message: (messages as string)[0],
          });
        });
      }
    },
  });
};

return (
  <form onSubmit={handleSubmit(onSubmit)} className="space-y-8">
    { /* — Tiêu đề — */ }
    <div>
      <label className="block text-sm font-medium mb-1">
        Tiêu đề công thức <span className="text-red-500">*</span>
      </label>
      <input
        {...register('title')} // register trả về ref, name, onChange,
onBlur
        placeholder="VD: Phở bò truyền thống Hà Nội"

```

```

        className={`input ${errors.title ? 'input-error' : ''}`}
      />
      {/* Hiển thị lỗi từ Zod validation */}
      {errors.title && <p className="text-red-500 text-sm mt-
1">{errors.title.message}</p>}
    </div>

    {/* — Độ khó — dùng Controller cho non-native inputs — */}
    <div>
      <label className="block text-sm font-medium mb-1">Độ khó</label>
      <Controller
        name="difficulty"
        control={control}
        render={({ field }) => (
          <select {...field} className="select">
            <option value="Easy">Dễ</option>
            <option value="Medium">Trung bình</option>
            <option value="Hard">Khó</option>
          </select>
        )}
      />
    </div>

    {/* — Dynamic Steps — */}
    <div>
      <h3 className="text-lg font-semibold mb-3">Các bước thực hiện</h3>
      {stepFields.map((field, index) => (
        <div key={field.id} className="flex gap-3 mb-3">
          <span className="flex-shrink-0 font-bold text-orange-500">{index +
1}</span>
          <div className="flex-1">
            <textarea
              {...register(`steps.${index}.description`)}
              placeholder={`Bước ${index + 1}: Mô tả chi tiết...`}
              rows={2}
              className="textarea w-full"
            />
            {errors.steps?.[index]?.description && (
              <p className="text-red-500 text-
sm">{errors.steps[index]?.description?.message}</p>
            )}
          </div>
          <button type="button" onClick={() => removeStep(index)}
            className="text-red-400 hover:text-red-600">X</button>
        </div>
      )})}
      {errors.steps?.root && <p className="text-red-500 text-
sm">{errors.steps.root.message}</p>}
      <button type="button" onClick={() => addStep({ description: '' })}
        className="btn btn-outline">+ Thêm bước</button>
    </div>

    {/* Submit */}
    <button type="submit" disabled={isPending || isSubmitting} className="btn
btn-primary w-full">
      {isPending ? 'Đang lưu...' : 'Đăng công thức'}
    </button>
  </form>
);

```

}

☑ Mẹo

useFieldArray() tự quản lý key cho list items — không cần tự tạo UUID. field.id là stable key được tạo bởi React Hook Form. Luôn dùng fields.map(field => <div key={field.id}> thay vì key={index} để tránh re-render issues khi remove/reorder.

4.7. GLOBAL STATE MANAGEMENT VỚI ZUSTAND

Sau khi có TanStack Query quản lý server state, câu hỏi đặt ra: còn lại loại state nào cần Zustand? Câu trả lời: Client State — những state không đến từ server: trạng thái UI (modal mở/đóng, sidebar, dark mode), user preferences, navigation history. Zustand là lựa chọn tối giản và performant nhất cho loại này.

Loại State	Ví dụ trong Culinary Blog	Dùng gì?	Lý do
Server State	Recipe list, recipe detail, comments	TanStack Query	Caching, background refetch, stale detection
Client State — UI	Sidebar mở/đóng, modal active, loading overlay	Zustand	Không cần caching, thay đổi thường xuyên
Client State — Persist	View mode (grid/list), theme preference	Zustand + persist	Lưu vào localStorage qua middleware
Form State	Create Recipe form values, validation errors	React Hook Form	Tối ưu cho forms: no re-render khi gõ
URL State	Search query, page, category filter	URL searchParams	Shareable, bookmarkable, browser back/forward
Local Component State	Dropdown hover, tooltip visible	useState	Không chia sẻ với component khác

Bảng 4.3. Phân loại State và công cụ quản lý phù hợp

TypeScript store/useUIStore.ts

```
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

// — Type định nghĩa rõ ràng cho toàn bộ store —————
interface UIState {
  // Sidebar navigation
  sidebarOpen: boolean;
  toggleSidebar: () => void;
  setSidebarOpen: (open: boolean) => void;

  // Modal system — chỉ một modal active tại một thời điểm
```

```

    activeModal: string | null;
    openModal: (id: string) => void;
    closeModal: () => void;

    // User preferences – được persist vào LocalStorage
    recipesViewMode: 'grid' | 'list';
    setRecipesViewMode: (mode: 'grid' | 'list') => void;
  }

  // create() với persist middleware – các preferences sống qua page reload
  export const useUIStore = create<UIState>()(
    persist(
      (set) => ({
        // — Sidebar —————
        sidebarOpen: false,
        toggleSidebar: () => set(state => ({ sidebarOpen: !state.sidebarOpen })),
        setSidebarOpen: (open) => set({ sidebarOpen: open }),

        // — Modal —————
        activeModal: null,
        openModal: (id) => set({ activeModal: id }),
        closeModal: () => set({ activeModal: null }),

        // — View Mode —————
        recipesViewMode: 'grid',
        setRecipesViewMode: (mode) => set({ recipesViewMode: mode }),
      }),
      {
        name: 'culinary-blog-ui-v1', // localStorage key
        // partialize: chỉ persist một phần state – không persist modal/sidebar
        partialize: (state) => ({
          recipesViewMode: state.recipesViewMode,
        }),
      }
    )
  );

  // — Selector patterns – tránh re-render không cần thiết —————
  // Chỉ subscribe vào đúng slice cần thiết
  // export const useSidebarOpen = () => useUIStore(s => s.sidebarOpen);
  // export const useActiveModal = () => useUIStore(s => s.activeModal);
  // Dùng useShallow khi select multiple values từ store
  // import { useShallow } from 'zustand/react/shallow';
  // const { open, close } = useUIStore(useShallow(s => ({ open: s.openModal,
  //   close: s.closeModal })));

```

4.8. NEXT.JS IMAGE OPTIMIZATION

Ảnh thường chiếm 60–80% tổng bandwidth của trang web. Ảnh lớn không được tối ưu là nguyên nhân hàng đầu làm giảm Core Web Vitals, đặc biệt LCP (Largest Contentful Paint). Next.js Image component (<Image>) tự động tối ưu ảnh: chuyển sang WebP/AVIF, lazy load, resize theo viewport, và sinh blur placeholder.

Tính năng	 HTML thông thường	Next.js <Image>
Lazy Loading	Không (cần thêm loading="lazy")	☑ Tự động (trừ priority)
Format tối ưu	Giữ nguyên format gốc	☑ WebP (50% nhỏ hơn JPEG), AVIF (60% nhỏ hơn)
Responsive sizes	Cần code thủ công	☑ Tự resize theo sizes prop
Blur placeholder	Không hỗ trợ	☑ placeholder="blur" — UX tốt hơn
Layout shift (CLS)	Dễ bị CLS nếu không set width/height	☑ Tự động prevent CLS
Priority loading	Không kiểm soát được	priority prop: preload LCP image
Remote images	Tự do	⚠ Phải whitelist domain trong next.config.ts

Bảng 4.4. So sánh `<img>` HTML và Next.js `<Image>` component

```
TypeScript/JSX components/recipe/RecipeCard.tsx

import Image    from 'next/image';
import Link     from 'next/link';
import type { RecipeListDto } from '@types/api';

interface RecipeCardProps { recipe: RecipeListDto; priority?: boolean; }

export function RecipeCard({ recipe, priority = false }: RecipeCardProps) {
  return (
    <Link href={`/${recipes}/${recipe.slug}`} className="group block">
      <article className="rounded-2xl overflow-hidden bg-white shadow-sm
hover:shadow-md transition">

        {/* Ảnh container – phải có position: relative khi dùng fill */}
        <div className="relative aspect-video">
          <Image
            src={recipe.primaryImageUrl ?? '/images/recipe-placeholder.jpg'}
            alt={`Ảnh công thức: ${recipe.title}`}
            fill // Chiếm toàn bộ container
            sizes=(max-width: 640) 100,
                  (max-width: 1024) 50,
                  33 // Hint cho browser về kích thước hiển thị
            className="object-cover group-hover:scale-105 transition-transform
duration-300"
            placeholder="blur"
            // blurDataURL: ảnh placeholder 10x10px base64 – sinh từ build time
            // Hoặc dùng thư viện plaiceholder để tạo tự động
            blurDataURL="
/2wBDAAAgGBgcGBQgHBwcJCQgKDBQNDASLDBkSEw8UHRofHh0aHBwgJc4nICIsIxwckDcpLDAXND
Q0Hyc5PTgyPC4zNDL/2wBDAQkJCQwLDBgNDRgyIRwhMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyM
jIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyMjIyM
AAAAAAAAAAAAABgQF/8QAIAAAAQMEAgMAAAAAAAAAAAAAAQIDBAAFERihMUH/xAAVAQEBAAAAAA
AAAAAAAAAAAAACA/EABgRAQEBAQEAAAAAAAAAAAAAABEQIS/9oADAMBAAIRAxEAPwPJJOSx4
xeWCDTMhvD2lWtqO/YvSlp4xo3703rSShVGGE7u30CdvnPnAqBSp//2Q=="
            priority={priority} // true cho LCP image (ảnh đầu tiên thấy)
```

```

    />
    { /* Badge độ khó */}
    <span className="absolute top-2 right-2 px-2 py-1 text-xs font-medium
bg-white/90 rounded-full">
      {recipe.difficulty === 'Easy' ? 'Dễ' : recipe.difficulty ===
'Medium' ? 'Trung bình' : 'Khó'}
    </span>
  </div>

  <div className="p-4">
    <h3 className="font-semibold text-gray-900 line-clamp-2 mb-
1">{recipe.title}</h3>
    <p className="text-sm text-gray-500 line-clamp-2 mb-
3">{recipe.description}</p>
    <div className="flex items-center gap-3 text-sm text-gray-400">
      <span>🕒 {recipe.prepTimeMinutes + recipe.cookTimeMinutes}
phút</span>
      <span>👤 {recipe.servings} người</span>
    </div>
  </div>
</article>
</Link>
);
}

```

TypeScript next.config.ts

```

import type { NextConfig } from 'next';

const nextConfig: NextConfig = {
  images: {
    // Whitelist các domain cho remote images
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'api.culinaryblog.vn', // Production API
        pathname: '/uploads/**',
      },
      {
        protocol: 'https',
        hostname: '**.minio.internal', // MinIO S3 (development)
      },
      {
        protocol: 'https',
        hostname: 'lh3.googleusercontent.com', // Google avatar (Social Login)
      },
    ],
    // Format ưu tiên: AVIF nhỏ hơn nhưng encode chậm hơn
    formats: ['image/avif', 'image/webp'],
  },
};

export default nextConfig;

```

4.9. CASE STUDY: RECIPE DETAIL VÀ COMMENT MODULE

Case Study tổng hợp toàn bộ Chương 4: xây dựng trang Recipe Detail với SSR (Server Component), tích hợp Comment Module với TanStack Query (Client Component), và bảo vệ route tạo recipe bằng Auth.js Middleware.

Bước 1 — API Types và Server-Side fetch helper

Text types/api.ts (subset cho Chương 4)

```
// Các types tương ứng với API response từ .NET backend
export interface RecipeListDto {
  id: string;
  title: string;
  slug: string;
  description?: string;
  primaryImageUrl?: string;
  prepTimeMinutes: number;
  cookTimeMinutes: number;
  servings: number;
  difficulty: 'Easy' | 'Medium' | 'Hard';
  categoryName: string;
  authorName: string;
  createdAt: string;
}

export interface RecipeDto extends RecipeListDto {
  instructions?: string;
  steps: RecipeStepDto[];
  ingredients: RecipeIngredientDto[];
  nutrition?: RecipeNutritionDto;
  likeCount: number;
  isLiked: boolean;
}

export interface CommentDto {
  id: string;
  content: string;
  authorName: string;
  authorAvatar?: string;
  createdAt: string;
}

export interface PaginatedResult<T> {
  items: T[];
  totalCount: number;
  page: number;
  pageSize: number;
  totalPages: number;
  hasNextPage: boolean;
  hasPreviousPage: boolean;
}

export type RecipeFilters = {
  page?: number;
  pageSize?: number;
  categoryId?: string;
  difficulty?: 'Easy' | 'Medium' | 'Hard';
  search?: string;
}
```

```
};
```

Bước 2 — CommentSection: Client Component với TanStack Query

TypeScript/JSX components/recipe/CommentSection.tsx

```
'use client';
import { useState } from 'react';
import { useSession } from 'next-auth/react';
import Image from 'next/image';
import { useInfiniteComments } from '@hooks/useComments';
import apiClient from '@lib/api/axios';
import { useQueryClient } from '@tanstack/react-query';
import { recipeKeys } from '@hooks/useRecipes';

interface CommentSectionProps { recipeId: string; }

export function CommentSection({ recipeId }: CommentSectionProps) {
  const { data: session } = useSession();
  const queryClient = useQueryClient();
  const [newComment, setNewComment] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  // Infinite scroll comments
  const {
    data,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage,
    isLoading,
  } = useInfiniteComments(recipeId);

  // Flat list from all pages
  const comments = data?.pages.flatMap(p => p.items) ?? [];

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!newComment.trim() || isSubmitting) return;

    setIsSubmitting(true);
    try {
      await apiClient.post(`/api/v1/recipes/${recipeId}/comments`, {
        content: newComment.trim(),
      });
      setNewComment('');
      // Invalidate comments cache → refetch từ đầu
      queryClient.invalidateQueries({ queryKey: recipeKeys.comments(recipeId) });
    } catch (error) {
      console.error('Lỗi khi gửi bình luận:', error);
    } finally {
      setIsSubmitting(false);
    }
  };

  if (isLoading) return <div className="animate-pulse h-32 bg-gray-100 rounded-x1" />;
```

```

return (
  <section className="mt-12">
    <h2 className="text-2xl font-semibold mb-6">
      Bình luận ({data?.pages[0]?.totalCount ?? 0})
    </h2>

    {/* Form thêm comment */}
    {session ? (
      <form onSubmit={handleSubmit} className="mb-8">
        <textarea
          value={newComment}
          onChange={(e) => setNewComment(e.target.value)}
          placeholder="Chia sẻ trải nghiệm của bạn về công thức này..."
          rows={3}
          className="w-full border rounded-xl p-3 resize-none"
          maxLength={1000}
        />
        <button type="submit" disabled={isSubmitting || !newComment.trim()}
          className="mt-2 btn btn-primary">
          {isSubmitting ? 'Đang gửi...' : 'Gửi bình luận'}
        </button>
      </form>
    ) : (
      <p className="mb-8 text-gray-500">
        <a href="/login" className="text-orange-500">Đăng nhập</a> để bình
luận
      </p>
    )}

    {/* Comment list */}
    <div className="space-y-4">
      {comments.map(comment => (
        <div key={comment.id} className="flex gap-3 p-4 bg-gray-50 rounded-
xl">
          <Image src={comment.authorAvatar ?? '/images/avatar.png'}
            alt={comment.authorName} width={40} height={40}
            className="rounded-full flex-shrink-0" />
          <div>
            <div className="font-medium">{comment.authorName}</div>
            <p className="text-gray-700 mt-1">{comment.content}</p>
            <time className="text-xs text-gray-400 mt-1 block">
              {new Date(comment.createdAt).toLocaleDateString('vi-VN')}
            </time>
          </div>
        </div>
      )})}
    </div>

    {/* Load more */}
    {hasNextPage && (
      <button onClick={() => fetchNextPage()} disabled={isFetchingNextPage}
        className="mt-6 w-full btn btn-outline">
        {isFetchingNextPage ? 'Đang tải...' : 'Xem thêm bình luận'}
      </button>
    )}
  </section>
);
}

```

Bước 3 — Setup và chạy project

Shell .env.local

```
# Next.js
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your-nextauth-secret-min-32-chars-random-string

# Backend API (đặt đúng port của .NET API)
NEXT_PUBLIC_API_URL=https://localhost:5001
API_URL=https://localhost:5001

# Google OAuth (lấy từ Google Cloud Console)
GOOGLE_CLIENT_ID=your-google-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=your-google-client-secret

# Next.js Image remote patterns cần biết origin
NEXT_PUBLIC_CDN_URL=https://cdn.culinaryblog.vn
```

Shell Terminal — Khởi tạo và chạy project

```
# Tạo Next.js project với TypeScript + Tailwind + App Router
npx create-next-app@latest culinary-blog-web \
  --typescript --tailwind --app --src-dir=false \
  --import-alias "@/*" --no-git

cd culinary-blog-web

# Cài đặt dependencies
npm install @tanstack/react-query @tanstack/react-query-devtools
npm install next-auth@beta # Auth.js v5 (next-auth v5)
npm install react-hook-form @hookform/resolvers zod
npm install zustand axios

# Dev server
npm run dev
# → http://localhost:3000
```

CÂU HỎI THẢO LUẬN

1. Giải thích sự khác biệt cốt lõi giữa React Server Components và Client Components về bundle size và khả năng fetch data. Trong Culinary Blog, trang nào nên là RSC và trang nào cần "use client"?
2. So sánh ISR và SSR: khi nào nên dùng ISR thay vì SSR cho trang Recipe Detail? Yếu tố nào quyết định (frequency of updates, personalization, SEO requirements)?
3. Tại sao TanStack Query dùng Query Keys là mảng thay vì string? Ưu điểm của query key hierarchy trong việc invalidation là gì?

4. Optimistic Update cải thiện UX như thế nào? Liệt kê 3 rủi ro có thể xảy ra với Optimistic Update nếu không implement đúng và cách TanStack Query xử lý chúng.
5. Auth.js lưu access token trong Session JWT (cookie) thay vì localStorage. Giải thích tại sao đây là lựa chọn bảo mật hơn. XSS attack ảnh hưởng khác nhau như thế nào đến hai cách lưu trữ này?
6. Zod `z.infer<typeof schema>` loại bỏ nhu cầu viết TypeScript interface riêng. Điều này đảm bảo gì về tính nhất quán giữa runtime validation và compile-time type checking?
7. Phân biệt Server State, Client State, URL State và Form State. Với mỗi loại, giải thích tại sao công cụ được đề xuất (TanStack Query, Zustand, URL params, React Hook Form) là phù hợp nhất.
8. Next.js `<Image>` tự động tạo các kích thước ảnh khác nhau dựa trên sizes prop. Giải thích cách browser chọn ảnh nào để hiển thị và tại sao điều này quan trọng cho Core Web Vitals.

BÀI TẬP THỰC HÀNH

Bài 1 — App Router Setup và Rendering Strategies (Cơ bản)

- Khởi tạo culinary-blog-web với create-next-app, cấu hình Route Groups, install tất cả dependencies.
- Tạo trang /recipes (ISR, revalidate 1 giờ) hiển thị danh sách recipes dưới dạng grid.
- Tạo trang /recipes/[slug] (SSR) hiển thị recipe detail với loading.tsx skeleton.
- Cấu hình next.config.ts cho remote images từ API server.
- Thêm generateMetadata() cho trang recipe detail với Open Graph tags.

Bài 2 — TanStack Query và Auth.js (Trung bình)

- Setup Providers với QueryClientProvider + SessionProvider, cấu hình QueryClient defaults.
- Implement Axios instance với request interceptor (gắn JWT) và response interceptor (handle 401).
- Viết custom hooks: useRecipes(), useRecipe(slug), useInfiniteComments(recipeId).
- Setup Auth.js v5 với Credentials Provider kết nối đến .NET API từ Chương 2.
- Cấu hình middleware.ts bảo vệ /dashboard và /recipes/new.

Bài 3 — Forms, Zustand và Tích hợp đầy đủ (Nâng cao)

- Xây dựng CreateRecipeForm hoàn chỉnh với Zod schema, dynamic field arrays cho steps và ingredients, error messages tiếng Việt.
- Implement useLikeRecipe() với Optimistic Updates: like → icon đỏ ngay → confirm từ server.
- Tạo useUIStore với Zustand: recipesViewMode (grid/list, persist), modal system.

- Tích hợp Social Login với Google vào Auth.js, test đăng nhập → nhận JWT backend.
- Performance audit: dùng Next.js Bundle Analyzer xem bundle size, đảm bảo Server Components không xuất hiện trong client bundle.

TÓM TẮT NỘI DUNG CỐT LÕI

Khái niệm	Mô tả	Trong Culinary Blog
App Router	File-based routing với /app directory, Route Groups, Layouts	(public), (auth), (dashboard) route groups
Server Component	Chạy server-only, fetch data trực tiếp, 0 bundle size	RecipeDetail, RecipesPage, layouts
Client Component	"use client" — hooks, events, browser API	CommentSection, CreateRecipeForm, UserMenu
ISR	SSG + revalidate mỗi N giây	recipes/page.tsx (revalidate: 3600)
SSR	Render mỗi request — luôn fresh, cần user context	recipes/[slug]/page.tsx
TanStack Query	Server state: cache, stale, refetch, deduplication	useRecipes, useRecipe, useInfiniteComments
Query Keys	Array uniquely identify cache entry → chính xác invalidation	recipeKeys.detail(slug)
Optimistic Update	UI update trước server → rollback nếu lỗi	useLikeRecipe onMutate/onError/onSettled
Auth.js v5	Session management: Credentials + Google, HTTP-only cookie	auth.ts, middleware.ts, useSession
Zod + z.infer	Schema = runtime validation + TypeScript type — single source	createRecipeSchema, CreateRecipeFormData
useFieldArray	Dynamic form fields (add/remove steps, ingredients)	CreateRecipeForm steps và ingredients
Zustand persist	Client state lưu qua reload — partialize chọn gì persist	useUIStore recipesViewMode
<Image> sizes	Hint browser về kích thước hiển thị → đúng image được tải	RecipeCard sizes prop

Từ khóa quan trọng:

App Router, Route Groups, Server Components, "use client", ISR (revalidate), SSR, Streaming, [useQuery](#) / [useMutation](#) / [useInfiniteQuery](#), Query Keys, Optimistic Updates,

stale-while-revalidate, `Auth.js auth()` (server) / `useSession()` (client), `middleware.ts`, `zodResolver`, `z.infer<typeof schema>`, `useFieldArray`, Client State vs Server State, `persist` middleware (Zustand), `<Image fill sizes priority>`

TÀI LIỆU THAM KHẢO MỞ RỘNG

Tài liệu kỹ thuật chính thức:

- Next.js Documentation — App Router: <https://nextjs.org/docs/app>
- Next.js Documentation — Data Fetching Patterns: <https://nextjs.org/docs/app/building-your-application/data-fetching>
- TanStack Query v5 Documentation: <https://tanstack.com/query/v5/docs/framework/react/overview>
- Auth.js v5 Documentation: <https://authjs.dev/getting-started>
- React Hook Form Documentation: <https://react-hook-form.com/docs>
- Zod Documentation: <https://zod.dev>
- Zustand Documentation: <https://docs.pmnd.rs/zustand/getting-started/introduction>

Bài viết và tài nguyên học tập:

- Theo Browne — Understanding React Server Components: <https://www.joshwcomeau.com/react/server-components/>
- TkDodo (Dominik D.) — Practical React Query: <https://tkdodo.eu/blog/practical-react-query> (series bài viết cực kỳ chi tiết về TanStack Query)
- Lee Rob (Vercel) — Next.js App Router Migration Guide: YouTube / Vercel blog
- Colby Fayock — Optimistic UI with TanStack Query: <https://www.freecodecamp.org/news/optimistic-ui-react-query/>
- Matt Pocock — Zod tutorial: <https://www.totaltypescript.com/tutorials/zod>